A Gold-Standard Study of What Makes a Lightweight Game-Playing Agent Strong

Anonymous submission

Abstract

Reinforcement learning agents for imperfect-information card games are only as strong as the opponents they train against, and they are hard to grade, since they beat a random opponent over 99 percent of the time and only tie copies of themselves. So we build a strong, fixed, rule-based expert for Gin Rummy and use it only as a yardstick, never for training. It beats every agent we trained 70 to 99 percent of the time. Across more than a hundred runs, we isolate what makes a lightweight agent stronger. Trust region updates, a well-aimed reward, a curriculum of tougher opponents, warm starting, and keeping the best checkpoint all help, and stacking them lifts a self-play champion from about 30 to 36 percent against the expert. Several ideas did not pay off. Short-term and longer-term reward shaping, learned state embeddings, imitation and DAgger, and a live large language model opponent were each unhelpful, too slow, or too heavy to train at scale. Comparing MLP, convolutional, set-based, attention, and recurrent encoders shows that extra capacity does little to break the ceiling, suggesting the limit is information rather than network size. We add standard baselines (neural fictitious self-play and information set Monte Carlo search) and confirm the approach carries over to Leduc Hold'em, where the optimum is computable. The result is a lightweight, game-agnostic recipe that trains competitive agents without training on the expert, for any game a small model can handle, reported with robust statistics and released as a reusable package.

Introduction

Learning to play imperfect-information card games well is a long-standing testbed for game AI. Games such as poker, DouDizhu, Mahjong, and Gin Rummy hide the opponent's hand and the order of the deck, so a player must act under uncertainty, plan over a long horizon, and adapt to whoever sits across the table. The dominant recipe for building agents in these games is deep reinforcement learning (RL) with self-play, which has produced superhuman play in several of them (Heinrich and Silver 2016; Li et al. 2020; Zha et al. 2021). Behind those headline results, though, a practitioner who wants a small and fast agent runs into two practical problems that the headline results do not solve.

The first is the *opponent bottleneck*. An RL agent is only as good as the opponents it practices against. Training against a weak fixed opponent teaches a weak ceiling, and pure self-play can chase its own tail, cycling through strategies that beat the last version without getting stronger in any absolute sense. The second problem is that, for most of these games, the practitioner has *no cheap absolute yardstick*. Strength is usually measured against random play or against the agent's own past checkpoints, both of which can flatter a mediocre agent. As a result, the design choices that actually build a strong lightweight agent (which RL algorithm, how to shape the reward, how to schedule opponents, how to represent the state, which checkpoint to ship) are tuned against references that are either too weak or constantly moving. The field has many strong single agents but few clean, controlled accounts of which of these choices matter and why. If we had a fixed, strong, and cheap reference opponent, we could measure these choices honestly and turn them into a reusable recipe.

Existing lines of work do not fill this gap directly. The famous self-play systems (Silver et al. 2018; Vinyals et al. 2019; Berner et al. 2019) answer what is possible with very large compute, not which lightweight choices matter on a single GPU. Search and equilibrium methods such as counterfactual regret minimization and its descendants (Zinkevich et al. 2007; Brown and Sandholm 2018; Brown et al. 2019) produce extremely strong play, but they are a different family of method (search over the game tree), and they do not tell you how to set the reward or curriculum for a compact reactive policy. Prior Gin Rummy work studies single methods in isolation, for example temporal-difference self-play (Kotnik and Kalita 2003) or hand-tuned heuristic and ensemble agents (Nagai et al. 2021), rather than a controlled head-to-head of training choices graded against one fixed expert.

We take a deliberately simple position. For Gin Rummy we build a strong, fixed, deterministic expert that solves the one subproblem with a clean optimum (meld decomposition, that is, the lowest-deadwood way to arrange a hand) and otherwise plays principled endgame moves. It is not a game-theoretic optimum for the full imperfect-information game, and we do not claim it is; what matters is that it is strong, reproducible, and cheap, so it makes an honest yardstick. We never train against it. We then run more than one hundred controlled experiments that change one factor at a time and grade every result against this fixed expert. This paper makes the following contributions.

- A reproducible gold-standard expert benchmark for Gin Rummy, and the empirical finding that strong play almost never gins. The expert gins in 0.7 to 1.7 percent of games and wins by knocking early with low deadwood. This single fact reframes how reward should be designed for the game.
- A controlled, apples-to-apples study, graded against the fixed expert, of which lightweight training choices actually help. Trust-region updates, a knock-first reward, a rising opponent curriculum, warm-starting, and keeping the best checkpoint each help; learned state embeddings, dense step rewards, imitation learning, and a live large-language-model (LLM) opponent each fail, each for an identified reason.
- The result that reward shaping cannot induce the human-intuitive but losing habit of chasing gin. Paying three times more for a gin than a knock leaves the gin rate under one percent, the same as not rewarding gin at all. We connect the parallel failure of imitation learning to causal confusion.
- Evidence that the ceiling is *information-bound, not capacity-bound*. Across MLP width and depth, convolutional, permutation-invariant set, attention, and recurrent encoders, win-rate against the expert stays in a narrow band with overlapping confidence intervals, and the ranking is robust across training recipes. A determinized information-set Monte-Carlo search, graded *fairly* (hidden cards re-dealt), is in fact weaker than our trained agents and loses to them head-to-head; an *oracle* variant that may see the hidden cards reaches far higher. The gap between the two quantifies the value of the hidden information, which is what sets the ceiling.
- A game-agnostic release of the pipeline, with the method reproduced on a second game, Leduc Hold'em, where a counterfactual-regret optimum is computable: a tabular learner graded against that optimum reaches near parity, showing the expert-yardstick study is not tied to Gin Rummy.

Related Work

Deep RL for imperfect-information card games. Self-play deep RL has mastered several hidden-information card and tile games. Neural Fictitious Self-Play approximates a Nash equilibrium by mixing a best-response network with an average-policy network (Heinrich and Silver 2016). Suphx reached expert human level at Mahjong with global reward prediction and oracle guiding (Li et al. 2020), and DouZero mastered the three-player game DouDizhu by combining deep networks with parallel Monte-Carlo self-play (Zha et al. 2021). RLCard provides a common toolkit and environments for this line of work, including Gin Rummy (Zha et al. 2020). Within Gin Rummy specifically, Kotnik and Kalita (2003) compared temporal-difference self-play with coevolution, and the EAAI undergraduate challenge produced strong hand-tuned and ensemble agents (Nagai et al. 2021). These works each introduce a single strong agent. We differ in goal: rather than proposing one more agent, we hold a fixed expert constant and use it to measure, in controlled experiments, which training choices make a lightweight agent stronger.

Self-play, leagues, and opponent curricula. AlphaGo Zero and AlphaZero showed that pure self-play with search can reach superhuman play in perfect-information games (Silver et al. 2017, 2018). AlphaStar added a league with prioritized fictitious self-play (PFSP), sampling harder opponents more often (Vinyals et al. 2019), and OpenAI Five scaled self-play to a complex video game (Berner et al. 2019). Curriculum learning more broadly orders training from easy to hard (Bengio et al. 2009), and a large literature studies curricula for RL (Narvekar et al. 2020). We adopt a small opponent curriculum and a PFSP variant, but our question is not how far self-play scales; it is which curriculum and checkpoint choices matter at small scale, measured against a fixed expert rather than against the moving target of self-play.

Search and equilibrium solving. Counterfactual regret minimization (Zinkevich et al. 2007) and its deep variants (Brown et al. 2019) underpin the superhuman poker agents Libratus and Pluribus (Brown and Sandholm 2018, 2019), and OpenSpiel collects many such algorithms and small benchmark games (Lanctot et al. 2019). Determinized search, or perfect-information Monte-Carlo, samples the hidden state and searches the resulting perfect-information games; information-set Monte-Carlo tree search (ISMCTS) is the canonical form (Cowling et al. 2012; Long et al. 2010). We use these as *baselines* rather than as our method: we run a determinized ISMCTS for Gin Rummy, and NFSP and CFR on a second game, and we use the family to frame the ceiling we observe. A key point we make precise later is that a determinized search graded *fairly* (without seeing the hidden cards) is far weaker than the same search given oracle access.

Network architectures for card observations. A hand is an unordered set of cards, which invites permutation-invariant encoders such as Deep Sets (Zaheer et al. 2017) and self-attention (Vaswani et al. 2017); convolutional stacks and recurrent networks (Hochreiter and Schmidhuber 1997) are the other standard choices for structured or sequential inputs. We hold our training recipe fixed and sweep these encoders to ask whether representation, rather than reward or curriculum, is the lever that breaks the ceiling.

Reward shaping and imitation learning. Potential-based reward shaping preserves the optimal policy while changing the learning signal (Ng et al. 1999); we use this view to interpret why some shaped rewards help and others hurt. DAgger reduces imitation learning to no-regret online learning by querying an expert on states the student visits (Ross et al. 2011). We find that imitation collapses here, and we attribute the collapse to causal confusion, the phenomenon in which a cloned policy latches onto spurious correlates of the expert action and breaks under distribution shift (de Haan et al. 2019).

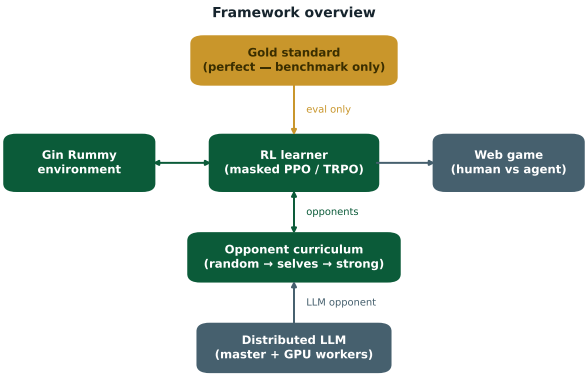


Figure 1: System overview. A masked PPO or TRPO learner trains against a rising curriculum of opponents (random play, a growing pool of past checkpoints, then self-play), with a distributed LLM available as an optional opponent. The fixed expert sits outside the training loop and is used only to grade agents, so no trained agent ever practices against the yardstick it is measured with.

Language models as game players. Cicero combined a language model with planning and RL to reach human-level play in Diplomacy, a game built on negotiation (Bakhtin et al. 2022). Motivated by such results, we test a modern instruction-tuned LLM (Qwen Team 2024) as a live opponent inside the training loop. We find it plays competently but is far too slow to serve the millions of moves that RL needs, which is itself a useful negative result for anyone considering live LLM opponents.

Action masking and tooling. Invalid-action masking is the standard way to keep a policy-gradient agent inside the legal action set (Huang and Ontañón 2022). We build on PettingZoo for the multi-agent interface (Terry et al. 2021), Stable-Baselines3 for PPO and TRPO (Raffin et al. 2021; Schulman et al. 2017, 2015, 2016), and a paged-attention server for LLM inference (Kwon et al. 2023).

Problem Setup

We study two-player Gin Rummy as implemented in RLCard and exposed through PettingZoo as `gin_rummy_v4` (Zha et al. 2020; Terry et al. 2021). Figure 1 shows how the pieces fit together: a masked RL learner, a curriculum of opponents, and the fixed expert that grades agents without ever entering training. Each turn a player draws (from the stock or the discard pile), then discards, and may knock when the deadwood (the point value of cards left outside melds) is at most ten, or declare gin at zero deadwood. Knocking with low deadwood wins a small score; gin wins a larger bonus but requires a complete hand and so is riskier.

Observation and actions. The agent sees a 4×52 binary tensor: its own hand, the top of the discard pile, the other visible discards, and the set of unknown cards (in the opponent's hand or the stock). It never sees the opponent's hand. The action space has 110 discrete actions (draw, pick up, declare, and one discard or knock per card), and at every step the environment supplies a binary mask of the legal actions.

Single-agent reduction. We turn the two-player game into a single-agent learning problem: one seat is the learner and the other is an opponent drawn from a curriculum (described below). The wrapper steps the opponent to completion between the learner's decisions and rotates seats across games so that neither position is favored. This reduction is what lets us swap in any opponent, including the fixed expert for evaluation and the LLM for the live-opponent test, without changing the learner.

Masked actor-critic. Our policy is a masked actor-critic shared by PPO and TRPO (Schulman et al. 2017, 2015). We compute action logits, then set the logits of illegal actions to a large finite negative value before the softmax. We use a large finite value rather than negative infinity on purpose: PPO tolerates infinities, but the conjugate-gradient and KL computations in TRPO produce NaNs on them, so a large finite fill gives near-zero probability to illegal moves while keeping the same network usable by both algorithms. At evaluation we always take the highest-probability *legal* action. We never fall back to a random legal move, because a random fallback would quietly inflate measured strength by hiding states where the policy is undecided.

The Gold-Standard Expert

The center of our method is a fixed reference opponent that is strong, cheap, and fully reproducible. We build it from one exact component and a few principled heuristics, and we are precise about which is which.

Exact melding. Given a hand, the lowest achievable deadwood is a well-defined optimization: enumerate the valid ways to group cards into runs and sets, and keep the arrangement with the least leftover value. We solve this exactly using RLCard's meld enumeration, memoized over hands so that the whole benchmark runs quickly. This is the one place where the expert is provably optimal, and it is optimal only for this subproblem, not for the full game.

Principled endgame. On top of exact deadwood, the expert draws the up-card only when doing so strictly lowers its best achievable deadwood, discards the card that minimizes resulting deadwood (breaking ties by shedding the highest-value card), and knocks as soon as it legally can, declaring gin only when gin is reachable without delay. These are sensible, deterministic rules. They are not tuned to the opponent and they do not reason about the opponent's hidden cards, so the expert is not a game-theoretic optimum.

Why this is the right yardstick. A game-theoretic optimum for imperfect-information Gin Rummy is not available at low cost, and we do not need one. What an honest study needs is a reference that is (i) strong enough to separate good agents from weak ones, (ii) identical in every comparison, and (iii) cheap enough to grade thousands of games. Our expert satisfies all three. As we show next, it beats every agent we trained by a wide margin, so it is a meaningful ceiling,

and because it is fixed and deterministic, a win-rate against it is directly comparable across every experiment in the paper. We use the expert only to score agents; it never appears in the training loop, so no result below is contaminated by training against the thing we measure with.

Training Methods Studied

All learning agents share the masked actor-critic above. Around it we vary one ingredient at a time. This section describes the ingredients; the next two describe how we evaluate them and what we found.

Algorithm. We compare PPO, which clips the policy update, with TRPO, which bounds the update by a trust region on the policy's KL divergence (Schulman et al. 2017, 2015). Both use generalized advantage estimation (Schulman et al. 2016) and the same network, curriculum, and reward, so any difference is the algorithm.

Reward shaping. The native reward is sparse: a score at the end of the game. We study shaped rewards that change the relative payoff of knocking and ginning, and a small dense bonus for reducing one's own deadwood. We read these through the lens of potential-based shaping (Ng et al. 1999): a shaping term that tracks real progress should help, while one that rewards a proxy can pull the agent off the true objective.

Opponent curriculum. We schedule opponents in three stages: first random play, then a growing pool of past checkpoints, then a mix that adds self-play. The pool can be sampled uniformly over recent checkpoints or with a PFSP weighting that practices more against opponents the agent is currently losing to (Vinyals et al. 2019). The stage boundaries are set as fractions of the training budget and synchronized across parallel workers through a small shared state file.

Warm-start and keep-the-best. We optionally warm-start a run from a strong earlier agent rather than from scratch. During training we evaluate against the hardest reference every fixed number of steps and save a copy whenever the agent improves, then ship that best checkpoint instead of the final one. The motivation, which the results confirm, is that training in a shifting self-play environment often drifts past its own peak.

State representation. The native observation is a sparse 4×52 tensor. We test whether a compact dense embedding helps by squeezing it into a small vector two ways: one learned to place game states that lead to similar outcomes near each other, and one where an LLM judges state similarity and we fit an embedding to those judgments. We test several embedding sizes and both frozen and unfrozen variants.

Imitation and dense supervision. We test DAgger, which trains the student to copy the expert's action on the states the student actually visits (Ross et al. 2011), and a short-horizon dense reward that scores each move by agreement with an evaluator over the next L steps. Both are popular shortcuts for injecting expert knowledge.

Live LLM opponent. We test whether a modern LLM can serve as a strong in-the-loop opponent. Because a single RL run issues tens of thousands of opponent queries and a 7B model answers in seconds, a naive loop would take far too long, so we built a distributed serving stack: a CPU master with a response cache that canonicalizes suit-equivalent positions into one cache key, and a pool of GPU workers that self-register and are health-checked and load-balanced by the master (Kwon et al. 2023; Qwen Team 2024). One engineering finding from this stack is reported with the results, because it matters for anyone trying the same thing: where the model weights are stored dominates worker start-up time.

Network architecture. Holding the best recipe fixed, we vary *only* the policy network and retrain from scratch (a different-shaped network cannot warm-start from the MLP champion, so this is a clean relative ranking at a fixed budget). We sweep MLP width and depth, a convolutional stack over the card planes, a permutation-invariant Deep Sets encoder (Zaheer et al. 2017), a self-attention encoder over card tokens (Vaswani et al. 2017), an LSTM (Hochreiter and Schmidhuber 1997), activation functions, and weight decay. To test whether any gain is recipe-specific, we also rerun the top encoders under two alternative recipes (PPO in place of TRPO, and PFSP sampling).

Search and learning baselines. We add two non-learned and learned references. The first is a determinized information-set Monte-Carlo search (ISMCTS/PIMC) (Cowling et al. 2012; Long et al. 2010) that, before each rollout, re-deals the cards it cannot see (the opponent's hand and the stock) uniformly from the unseen pool, so it never uses the true hidden cards; we vary the per-move rollout budget. For contrast we also run an *oracle* variant that searches from the true state, as an upper bound. The second is Neural Fictitious Self-Play (Heinrich and Silver 2016), the canonical equilibrium-learning baseline, run through OpenSpiel (Lanctot et al. 2019).

A second game. To check that the method is not tied to Gin Rummy, we repeat the expert-yardstick study on Leduc Hold'em, a small poker game whose game-theoretic optimum is computable. We solve a near-optimal expert with counterfactual regret minimization (Zinkevich et al. 2007; Lanctot et al. 2019) and grade a tabular self-play learner and NFSP against it, the exact analogue of grading our Gin Rummy agents against the fixed expert.

Experimental Setup

One metric, applied the same way everywhere. Every agent is graded by its win-rate against the fixed expert, with seats rotated so that each agent plays both positions equally. We use this single metric because the expert is strong, identical across comparisons, and cheap, so a win-rate against it isolates the effect of whatever ingredient we changed. We report two supporting numbers where useful: win-rate against the prior self-play champion (a moving but familiar reference) and against random play (a floor that saturates). For

the headline agents we run 2000 games and report a 95 percent confidence interval; for the broad sweeps we run 400 to 600 games per cell, which is enough to rank regimes. For multi-seed comparisons such as the architecture sweep we follow current guidance on RL evaluation and report the interquartile mean (IQM) with stratified bootstrap confidence intervals rather than a single best run (Agarwal et al. 2021; Henderson et al. 2018).

Protocol. Each study changes one factor and holds the rest fixed. Algorithm runs use two random seeds at two million steps. The reward and curriculum sweep changes one setting at a time from a common baseline. Representation runs hold the trainer fixed and swap only the input. Evaluation always uses the highest-probability legal action, never a random fallback, for the reason given above. Win-rates count a positive game score as a win.

Results

The expert is strong, and it almost never gins

The expert beats every agent we trained between 70 and 99 percent of the time: for example 70.2 percent against the self-play champion and 99.5 percent against random play (Figure 2, left). The surprise is on the right of Figure 2: despite gin scoring the most points, the expert gins in only 0.7 to 1.7 percent of games. It wins by knocking early with low deadwood. Chasing gin, the move a beginner reaches for, is a trap. The best style is patient, low-risk knocking. Every reward result below should be read with this in mind.

Algorithm: trust-region updates win

With the same masking, curriculum, and reward at two million steps, TRPO beats PPO on every opponent and both seeds. Against the expert, TRPO reaches 22.5 percent and PPO 15.0 percent; against the champion, 50.5 percent versus 31.0 percent (Figure 3, left). The reason fits the setting: rewards here are rare and the opponent keeps changing, and a trust region takes smaller, safer steps that do not overreact to a lucky or unlucky batch. We note that some popular alternatives do not apply here: methods like GRPO and DPO are built for aligning language models from preference data, not for per-move control in a game environment, so they are out of scope.

Reward: you cannot pay the agent into ginning

The gin-to-knock payoff is a real dial for *style*: when we reward gin heavily the agent gins more often against weak opponents, and when we reward knocking it knocks almost always. Against random play, a knock-first reward yields 97 percent knocks at a 99.4 percent win-rate, and a gin-first reward yields 22 percent gins at a 98.3 percent win-rate. But style is not strength. When graded against the expert, the gin rate collapses no matter how we set the reward. Across the controlled sweep, paying three times more for a gin than a knock leaves the gin rate under one percent, statistically the same as not rewarding gin at all (Figure 3, right). Just like the expert, the agent works out on its own that against strong play, chasing gin loses. This is the cleanest result in the paper: a shaped reward can set the agent's style, but it cannot bribe the agent into a habit that loses.

Table 1: Win-rate against the fixed expert for the main milestones (higher is better), with win-rate against the prior self-play champion for context. The headline agent is evaluated over 2000 games with a 95 percent confidence interval; the others over 400 to 600 games. Tuning the algorithm, reward, curriculum, and checkpointing tops out in the mid-thirties against the expert.

Regime	vs. expert	vs. champion
PPO baseline, 2M steps	15.0	31.0
TRPO baseline, 2M steps	22.5	50.5
Self-play champion (prior best)	∼30	(self)
Stacked recipe (this work)	**34.2 ± 2.1**	**51.4**

Representation: the raw sparse input wins

Compressing the sparse 4×52 observation into a small dense embedding hurt in every variant we tried. Against the expert, the raw sparse input reaches about 15 percent, while learned and LLM-judged embeddings land between 6 and 14 percent depending on size, and never above the sparse baseline. Larger embeddings and unfreezing the encoder did not close the gap. The reason is that a fixed low-dimensional bottleneck is good at saying whether two states are broadly similar, but it discards the fine detail (which exact cards are where) that the policy needs to act well.

Curriculum, warm-start, and keep-the-best

Self-play makes difficulty free: a three-million-step self-play agent beats its own parent 61 percent of the time. But unguided self-play is fragile; a pool of past selves with no schedule broke down after about ten million steps as the agents chased circular strategies. A rising curriculum (random, then a pool, then self-play) keeps the challenge fair but always a little harder, and avoids that collapse. Two cheap additions matter on top of it. Warm-starting from the champion starts a run strong and lets it specialize. Keeping the best checkpoint matters most: because training drifts past its peak, evaluating every million steps and shipping the best checkpoint recovers two to three points for free (Figure 4). A more elaborate opponent-picking scheme (PFSP) was about even with a simple schedule here, and a longer reward horizon (a higher discount) slightly hurt, adding noise rather than foresight.

Stacking the ingredients that help (TRPO, a knock-first reward with a small deadwood-reduction bonus, a curriculum seeded with the strongest earlier agents, warm-starting, and keep-the-best) and checking over 2000 games, the strongest agent reaches $\mathbf{34.2 \pm 2.1}$ percent against the expert and 51.4 percent against the old champion, up from the champion's roughly 30 percent (Table 1). Two sibling agents trained with PFSP reach 34.0 percent, statistically the same. Table 2 summarizes every ingredient and its verdict.

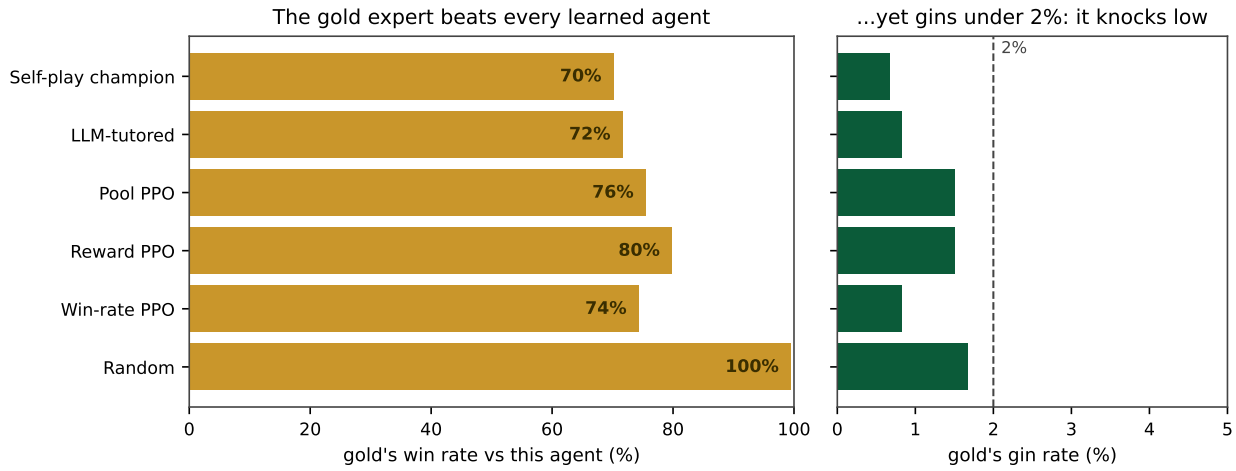


Figure 2: The fixed expert beats every agent we trained (left) yet gins in under two percent of games (right). It wins by knocking early with low deadwood, not by chasing gin. This is the observation that reframes reward design for the game.

The honest negatives

Three popular shortcuts each failed, and the failures are as informative as the successes.

Imitation learning (DAgger). Copying the expert's moves one at a time drove the training loss to near zero but produced an agent that wins almost no games. This is a textbook case of causal confusion (de Haan et al. 2019): the student learns to reproduce the expert's move in familiar positions without learning the reasoning behind it (tracking what the opponent is collecting), so it falls apart the moment the game leaves the demonstrated distribution. Copying a move is not the same as understanding why.

Dense step rewards. Scoring every move against an evaluator over a short horizon made the agent short-sighted. A horizon of two showed no real learning, and a horizon of five stopped improving after about five hundred thousand steps, at which point the agent farmed instant points instead of trying to win. Dense shaping pulled the policy away from the real, sparse goal, exactly the failure mode that the theory of reward shaping warns about when the bonus is not aligned with the true return (Ng et al. 1999).

Live LLM opponent. Modern instruction-tuned LLMs played Gin Rummy competently with a chain-of-thought prompt, reaching 98.2 percent legal moves (against 79.3 percent with a terse prompt), and one even beat our self-play agent three games to two in a short match. But at 9 to 27 seconds per move they are far too slow to supply the millions of moves an RL run needs; training this way would take weeks. A vision-language variant fed the board as an image failed outright at roughly ten percent legal moves, confirming that this is a text task, not an image task. The serving stack did expose one transferable engineering lesson: loading a 7B worker from networked home storage ran at about eleven megabytes per second and took roughly twenty-eight minutes, blowing the health-check timeout, while staging the same weights on fast scratch storage cut start-up to about twenty-seven seconds, a 62× speedup, after which a fourteen-worker pool sustained about thirty-two queries per second. The LLM is strong enough to be interesting and too slow to be useful as a live opponent.

Architecture, Baselines, and a Second Game

Having tuned the algorithm, reward, curriculum, and checkpointing to the mid-thirties, we ask the natural follow-up: is the network the missing lever, and how do principled baselines compare? The answer, in both cases, points at the same conclusion: the ceiling is set by the hidden information, not by the model.

Architecture does not break the ceiling

We hold the winning recipe fixed and vary only the network, retraining from scratch. Across MLP width and depth, convolutional, permutation-invariant Deep Sets, self-attention, and recurrent encoders, the win-rate against the expert stays in a narrow band: the best IQMs are the structured encoders (convolutional 31.1 percent, Deep Sets 30.4 percent) and the plain MLP anchor is 26.7 percent, but the 95 percent bootstrap intervals overlap throughout (Figure 5, left), so the architectures are statistically indistinguishable. Making the MLP wider, deeper, or extra-wide does not help, and weight decay slightly hurts. The finding is robust to the training recipe: rerun under PPO instead of TRPO, or with PFSP sampling, the encoders keep the same ordering and the best cell reaches 30.2 percent, in the same band. Recurrence (an LSTM) beats its matched feed-forward control (24.3 versus 19.2 percent) but both trail the TRPO anchor, so memory helps relative to its own baseline without breaking the ceiling. Capacity and inductive bias are not the bottleneck.

A fair search is weak; only an oracle beats the expert

We then run a determinized ISMCTS, the classic non-learned reference for hidden-information card games.

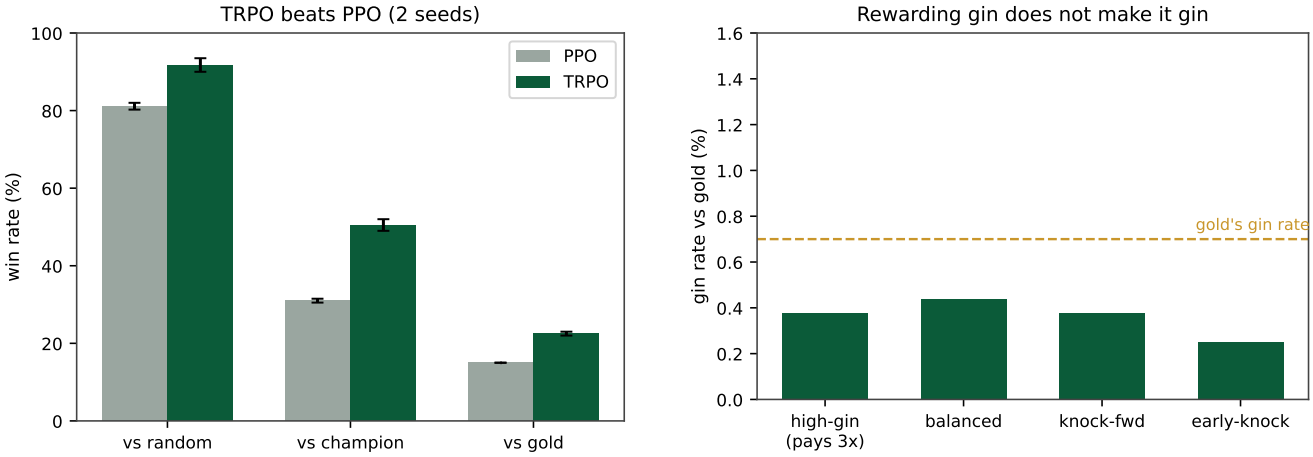


Figure 3: Left: TRPO beats PPO against every opponent. Right: the gin rate stays under one percent for every reward design we tried, including one that pays three times more for a gin than a knock. The dashed line marks the expert's own gin rate. You cannot pay the agent into chasing gin.

Table 2: Which ingredients moved the needle against the fixed expert, across more than one hundred controlled runs. Verdicts are measured by win-rate against the expert; the "why" column gives the mechanism we observe.

Ingredient	Verdict	Why
Keep the best checkpoint	**helps**	training drifts past its peak, so saving the best recovers two to three points
Warm-start from the champion	**helps**	start strong, then specialize, rather than relearn from scratch
TRPO over PPO	**helps**	smaller, safer steps suit rare rewards and a shifting opponent
Reward knocking, not gin	**helps**	matches the expert's low-risk, knock-early style
Rising opponent curriculum	**helps**	always a fair but slightly harder challenge; avoids self-play collapse
Pay three times more for gin	**no effect**	the agent refuses the losing habit at any payoff; gin rate stays under one percent
PFSP opponent picking	**no effect**	about even with a simple schedule at this scale
Learned state embeddings	**fails**	a fixed low-dimensional bottleneck throws away detail the policy needs
Imitation learning (DAgger)	**fails**	causal confusion: copies moves, not the reasoning behind them
Dense short-horizon rewards	**fails**	short-sighted: farms instant points, blind to winning
Live LLM opponent	**fails**	competent but 9 to 27 seconds per move, far too slow for RL-scale rollouts

Graded *fairly*, re-dealing the unseen cards before every rollout, it is surprisingly weak: win-rate against the expert rises with the per-move budget but only to 10, 17, 21, and 26 percent at 10, 30, 60, and 120 rollouts (Figure 5, right), *below* our trained agents. Gin Rummy hides a full hand plus the deck, so averaging over sampled deals is high-variance. Head to head against the fair search, every trained agent wins comfortably (66 to 69 percent at 60 rollouts; Table 3). For contrast, an oracle search that may see the hidden cards reaches 41 to 85 percent over the same budgets. The 26-versus-85 gap is the crux: the information, not the search, is what wins, which is direct evidence that the ceiling is information-bound. Naive search is not a free way past it.

Generality: a second game with a computable optimum

Finally we repeat the study on Leduc Hold'em, where a counterfactual-regret solution is computable (our CFR expert reaches exploitability 0.026). Graded against that near-optimal expert over eight seeds, a tabular self-play learner reaches near parity (mean return -0.085; random is about -0.78), reproducing the Gin Rummy story on a game where the true optimum is available. NFSP, the neural equilibrium baseline, was far more sample-hungry: even at three million episodes it averaged -0.71, near the random floor, an honest reminder that the neural baseline needs much more data than the tabular learner on this size of game. Table 3 collects the baseline and second-game numbers.

Discussion, Limitations, and Future Work

An information-bound ceiling. Stacking every choice that helps moves a lightweight agent from roughly 30 to about 34 percent against the expert (Table 1), and three independent lines of evidence say this ceiling is set by the hidden information, not by the method or the model. First, network architecture does not move it: MLP, convolutional, set-based, attention, and recurrent encoders are statistically indistinguishable, so capacity and inductive bias are not the bottleneck. Second, a determinized search graded fairly is *weaker* than our agents (26 percent), yet the same search

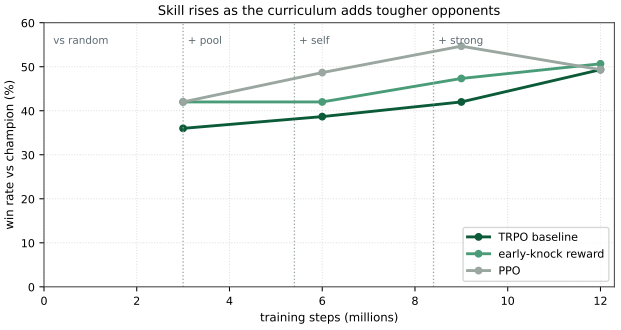


Figure 4: Win-rate against the champion as training moves through curriculum stages. The climb is real, but the late dip is why keeping the best checkpoint, rather than the last one, is worth two to three points.

Table 3: Baselines and the second game. Top: trained agents vs. the *fair* ISMCTS (model win-rate, 60 rollouts). Bottom: Leduc Hold'em mean return vs. the CFR-optimal expert (0 is parity, random ≈ -0.78).

Trained agent vs. fair ISMCTS	**win%**
Curriculum Ace (best)	69
League Tactician (PFSP)	69
Self-play champion	66
Leduc Hold'em vs. CFR optimum	**return**
Tabular Q self-play (8 seeds)	-0.085
NFSP, 3M episodes (4 seeds)	-0.71
Random baseline	-0.78

given oracle access to the hidden cards jumps to 85 percent; the gap is the value of the information. Third, our reactive policy has no memory and does not estimate the opponent's hand, which is exactly the information the oracle exploits. The ceiling is therefore a measurement, not a disappointment: it says that with the available information a lightweight reactive agent is already near the practical frontier, and that going clearly past it requires *more information* (opponent-hand inference) or much heavier search, not a bigger network or more reward tuning.

Limitations. We study one game, so the specific numbers are about Gin Rummy. Our expert is a strong heuristic, not a game-theoretic optimum, so "win-rate against the expert" is a fixed and meaningful yardstick rather than a distance from perfect play, and we have been careful to frame it that way. Our headline learner is a reactive feed-forward policy; we did test recurrence, but an LSTM helped only relative to its own control and did not break the ceiling, so a stronger form of memory or explicit opponent modeling is likely needed to track hidden information. And we tested the LLM only as a live opponent; we did not use it to generate an offline dataset, which our serving stack was in fact built to support.

Future work. The evidence points squarely at the hidden information. A naive determinized search did *not* break the ceiling at the budgets we tried, so the lever is not search alone but reasoning about what the opponent holds: explicit opponent-hand estimation (a thread already present in the Gin Rummy literature), belief-state or information-set search that conditions on inferred cards, and regret-minimization or equilibrium-finding methods (Zinkevich et al. 2007; Brown et al. 2019; Heinrich and Silver 2016) that target hidden information directly. Stronger memory than the LSTM we tested is a second axis. Finally, because our LLM serving stack can produce strong play offline even though it is too slow online, distilling a large batch of LLM-versus-LLM games into the policy with offline RL is a promising way to use the LLM without the latency that killed the live-opponent approach.

Beyond one game. Nothing in the method is specific to Gin Rummy. The recipe is a pattern: build a fixed strong reference for the game, grade every training choice against it, schedule a rising curriculum of opponents, and ship the best checkpoint rather than the last. We package the pipeline so that pointing it at another two-player game in the same multi-agent interface requires changing only the environment, and we hope the controlled, expert-graded style of study is useful to others building lightweight agents for interactive games.

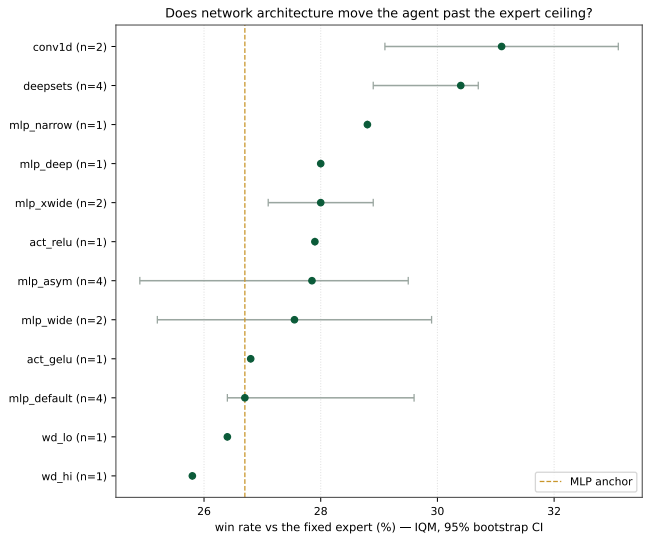
Conclusion

We set out to answer two questions for a small, fast Gin Rummy agent: how strong can it get, and which training choices make it strong. By building a fixed expert yardstick and grading more than one hundred controlled runs against it, we found a clear answer. Strong play knocks early and almost never gins, and no reward we tried could pay the agent into the losing habit of chasing gin. Trust-region self-play with a knock-first reward, a rising curriculum, warm-starting, and keeping the best checkpoint is the recipe that works, reaching about 34 percent against the expert; learned embeddings, imitation, dense rewards, and a live LLM opponent each fail for a reason we can name. Neither a smarter network nor a fair search breaks the ceiling: encoders are statistically indistinguishable, and a determinized search trails our agents while an oracle that peeks at the hidden cards soars, pinning the ceiling on the hidden information rather than the model. The same expert-yardstick study reproduces on Leduc Hold'em, where the optimum is computable. The biggest lesson is methodological: a fixed, strong, reproducible reference turns a pile of training tricks into a study with answers. We release the pipeline so the same study can be run on other games.

Reproducibility. Every figure is generated from saved evaluation logs, and the training curves are tracked online during runs. The training pipeline, the fixed expert, the distributed LLM serving stack, and the web client used for human play will be released as open source on acceptance.

References

Meta Fundamental AI Research Diplomacy Team (FAIR); Bakhtin, A.; Brown, N.; Dinan, E.; Farina, G.; Flaherty, C.;



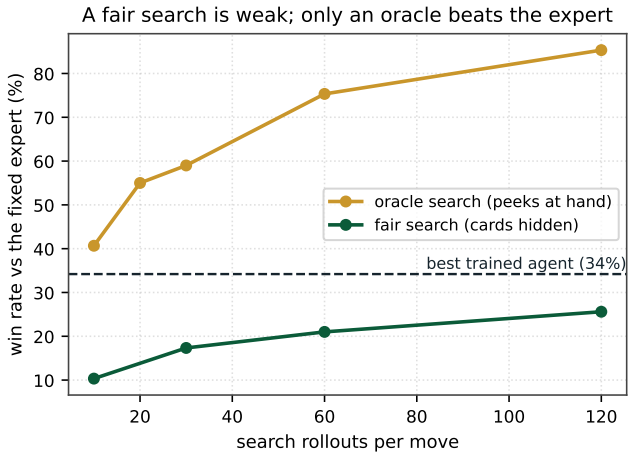


Figure 5: Left: win-rate against the fixed expert by network architecture (IQM with 95 percent stratified bootstrap CIs over seeds, from scratch at a fixed budget). Every encoder lands in a narrow band around the MLP anchor; the intervals overlap. Right: a determinized information-set search graded *fairly* (hidden cards re-dealt) tops out near 26 percent, below the trained agents, while the same search given *oracle* access to the hidden cards reaches 85 percent. The gap between the two curves is the value of the hidden information.

Fried, D.; Goff, A.; Gray, J.; Hu, H.; et al. 2022. Human-Level Play in the Game of *Diplomacy* by Combining Language Models with Strategic Reasoning. *Science* 378(6624): 1067–1074.

Agarwal, R.; Schwarzer, M.; Castro, P. S.; Courville, A. C.; and Bellemare, M. G. 2021. Deep Reinforcement Learning at the Edge of the Statistical Precipice. In *Advances in Neural Information Processing Systems (NeurIPS)* 34.

Bengio, Y.; Louradour, J.; Collobert, R.; and Weston, J. 2009. Curriculum Learning. In *Proceedings of the 26th International Conference on Machine Learning (ICML)*, 41–48.

Berner, C.; Brockman, G.; Chan, B.; Cheung, V.; Debiak, P.; Dennison, C.; Farhi, D.; Fischer, Q.; Hashme, S.; Hesse, C.; et al. 2019. Dota 2 with Large Scale Deep Reinforcement Learning. arXiv:1912.06680.

Brown, N.; and Sandholm, T. 2018. Superhuman AI for Heads-up No-Limit Poker: Libratus Beats Top Professionals. *Science* 359(6374): 418–424.

Brown, N.; and Sandholm, T. 2019. Superhuman AI for Multiplayer Poker. *Science* 365(6456): 885–890.

Brown, N.; Lerer, A.; Gross, S.; and Sandholm, T. 2019. Deep Counterfactual Regret Minimization. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 793–802.

Cowling, P. I.; Powley, E. J.; and Whitehouse, D. 2012. Information Set Monte Carlo Tree Search. *IEEE Transactions on Computational Intelligence and AI in Games* 4(2): 120–143.

de Haan, P.; Jayaraman, D.; and Levine, S. 2019. Causal Confusion in Imitation Learning. In *Advances in Neural Information Processing Systems (NeurIPS)* 32.

Henderson, P.; Islam, R.; Bachman, P.; Pineau, J.; Precup, D.; and Meger, D. 2018. Deep Reinforcement Learning That Matters. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence (AAAI)*, 3207–3214.

Hochreiter, S.; and Schmidhuber, J. 1997. Long Short-Term Memory. *Neural Computation* 9(8): 1735–1780.

Heinrich, J.; and Silver, D. 2016. Deep Reinforcement Learning from Self-Play in Imperfect-Information Games. arXiv:1603.01121.

Huang, S.; and Ontañón, S. 2022. A Closer Look at Invalid Action Masking in Policy Gradient Algorithms. In *Proceedings of the 35th International FLAIRS Conference*.

Kotnik, C.; and Kalita, J. 2003. The Significance of Temporal-Difference Learning in Self-Play Training: TD-rummy versus EVO-rummy. In *Proceedings of the 20th International Conference on Machine Learning (ICML)*, 369–375.

Kwon, W.; Li, Z.; Zhuang, S.; Sheng, Y.; Zheng, L.; Yu, C. H.; Gonzalez, J. E.; Zhang, H.; and Stoica, I. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*.

Lanctot, M.; Lockhart, E.; Lespiau, J.-B.; Zambaldi, V.; Upadhyay, S.; Pérolat, J.; Srinivasan, S.; Timbers, F.; Tuyls, K.; Omidshafiei, S.; et al. 2019. OpenSpiel: A Framework for Reinforcement Learning in Games. arXiv:1908.09453.

Li, J.; Koyamada, S.; Ye, Q.; Liu, G.; Wang, C.; Yang, R.; Zhao, L.; Qin, T.; Liu, T.-Y.; and Hon, H.-W. 2020. Suphx: Mastering Mahjong with Deep Reinforcement Learning. arXiv:2003.13590.

Long, J. R.; Sturtevant, N. R.; Buro, M.; and Furtak, T. 2010. Understanding the Success of Perfect Information Monte Carlo Sampling in Game Tree Search. In *Proceedings of the 24th AAAI Conference on Artificial Intelligence (AAAI)*, 134–140.

Nagai, M.; Shrivastava, K.; Ta, K.; Bogaerts, S.; and Byers, C. 2021. A Highly-Parameterized Ensemble to Play Gin Rummy. *Proceedings of the AAAI Conference on Artificial Intelligence* 35(17): 15614–15621 (EAAI Symposium).

Narvekar, S.; Peng, B.; Leonetti, M.; Sinapov, J.; Taylor, M. E.; and Stone, P. 2020. Curriculum Learning for Reinforcement Learning Domains: A Framework and Survey. *Journal of Machine Learning Research* 21(181): 1–50.

Ng, A. Y.; Harada, D.; and Russell, S. 1999. Policy Invariance under Reward Transformations: Theory and Application to Reward Shaping. In *Proceedings of the 16th International Conference on Machine Learning (ICML)*, 278–287.

Qwen Team. 2024. Qwen2.5 Technical Report. arXiv:2412.15115.

Raffin, A.; Hill, A.; Gleave, A.; Kanervisto, A.; Ernestus, M.; and Dormann, N. 2021. Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research* 22(268): 1–8.

Ross, S.; Gordon, G. J.; and Bagnell, J. A. 2011. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 627–635.

Schulman, J.; Levine, S.; Abbeel, P.; Jordan, M.; and Moritz, P. 2015. Trust Region Policy Optimization. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 1889–1897.

Schulman, J.; Moritz, P.; Levine, S.; Jordan, M.; and Abbeel, P. 2016. High-Dimensional Continuous Control Using Generalized Advantage Estimation. In *International Conference on Learning Representations (ICLR)*.

Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347.

Silver, D.; Schrittwieser, J.; Simonyan, K.; Antonoglou, I.; Huang, A.; Guez, A.; Hubert, T.; Baker, L.; Lai, M.; Bolton, A.; et al. 2017. Mastering the Game of Go without Human Knowledge. *Nature* 550(7676): 354–359.

Silver, D.; Hubert, T.; Schrittwieser, J.; Antonoglou, I.; Lai, M.; Guez, A.; Lanctot, M.; Sifre, L.; Kumaran, D.; Graepel, T.; et al. 2018. A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go through Self-Play. *Science* 362(6419): 1140–1144.

Terry, J. K.; Black, B.; Grammel, N.; Jayakumar, M.; Hari, A.; Sullivan, R.; Santos, L.; Perez, R.; Horsch, C.; Dieffendahl, C.; et al. 2021. PettingZoo: Gym for Multi-Agent Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)* 34.

Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, Ł.; and Polosukhin, I. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)* 30.

Vinyals, O.; Babuschkin, I.; Czarnecki, W. M.; Mathieu, M.; Dudzik, A.; Chung, J.; Choi, D. H.; Powell, R.; Ewalds, T.; Georgiev, P.; et al. 2019. Grandmaster Level in StarCraft II using Multi-Agent Reinforcement Learning. *Nature* 575(7782): 350–354.

Zaheer, M.; Kottur, S.; Ravanbakhsh, S.; Póczos, B.; Salakhutdinov, R.; and Smola, A. J. 2017. Deep Sets. In *Advances in Neural Information Processing Systems (NeurIPS)* 30.

Zha, D.; Lai, K.-H.; Huang, S.; Cao, Y.; Reddy, K.; Vargas, J.; Nguyen, A.; Wei, R.; Guo, J.; and Hu, X. 2020. RLCard: A Platform for Reinforcement Learning in Card Games. In *Proceedings of the 29th International Joint Conference on Artificial Intelligence (IJCAI)*, Demo Track.

Zha, D.; Xie, J.; Ma, W.; Zhang, S.; Lian, X.; Hu, X.; and Liu, J. 2021. DouZero: Mastering DouDizhu with Self-Play Deep Reinforcement Learning. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 12333–12344.

Zinkevich, M.; Johanson, M.; Bowling, M.; and Piccione, C. 2007. Regret Minimization in Games with Incomplete Information. In *Advances in Neural Information Processing Systems (NeurIPS)* 20.